Level - 1

01. Get a number from user and add 2 to that number and print the result.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter Number : ");
scanf("%d",&x);
y = x + 2;
printf("Result = %d",y);
}
```

Input :45 **Output** : 47.

02. Get a number from user and subtract 5 to that number and print the result.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter Number : ");
scanf("%d",&x);
y = x - 5;
printf("Result = %d",y);
}
```

Input :45 **Output** : 40.

03. Get a number from user and multiply 3 to that number and print the result.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter Number : ");
scanf("%d",&x);
y = x * 3;
printf("Result = %d",y);
}
```

Input :45 **Output** : 135.

04. Get a number from user and divide by the number by 6 and print the quotient.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter Number : ");
scanf("%d",&x);
y = x / 6;
printf("Result = %d",y);
}
```

Input :45 **Output** : 7.

05. Get a number from user and divide by the number by 8 and print the remainder.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter Number : ");
scanf("%d",&x);
y = x % 8;
printf("Result = %d",y);
}
```

Input :45 **Output** : 5.

06. Get a two-digit number from user and print the one’s digit.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = x % 10;
printf("One’s Digit NUmber is: %d",y);
}
```

Input :56 **Output** : 6.

07. Get a two-digit number from user and print the ten’s digit.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = x / 10;
printf("Ten’s Digit NUmber is: %d",y);
}
```

Input :45 **Output** : 4.

08. Get a three-digit number from user and print the one’s digit.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = x % 10;
printf("One's Digit NUmber is: %d",y);
}
```

Input :569 **Output** : 9.

09. Get a three-digit number from user and print the hundred’s digit.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = x / 100;
printf("Hundred’s Digit NUmber is: %d",y);
}
```

Input :569 **Output** : 5.

10. Get a three-digit number from user and print the ten’s digit.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = (x/10)%10;
printf("Ten's Digit NUmber is: %d",y);
}
```

Input :569 **Output** : 6.

11. Get a two-digit number from user and print sum the digits.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = (x/10) + (x%10);
printf("The Sum of the digits is: %d",y);
}
```

Input :56 **Output** : 11.

12. Get a three-digit number from user and print sum the digits.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = (x/100) + (x%10) + ((x/10)%10);
printf("The Sum of the digits is: %d",y);
}
```

Input :562 **Output** : 13.

13. Get a two-digit number from user and print the reverse of the number.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = (x/10) + ((x%10)*10);
printf("The Reverse of the Number is: %d",y);
}
```

Input :89 **Output** : 98.

14. Get a three-digit number from user and print the reverse of the number.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = (x/100) + (((x/10)%10)*10) + ((x%10)*100);
printf("The Reverse of the Number is: %d",y);
}
```

Input :895 **Output** : 598.

15. Get a four-digit number from user and only reverse the first two digits of the number, then print the number.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Four Digit Number : ");
scanf("%d",&x);
int first_two_digit, last_two_digit, reverse;
first_two_digit = x % 100;
last_two_digit = x / 100;
reverse = (first_two_digit/10) + ((first_two_digit %
10)*10);
y = (last_two_digit * 100) + reverse;
printf("Result =  %d",y);
}
```

Input :9561 **Output** : 9516.

16. Get a four-digit number from user and only reverse the last two digits of the number, then print the number.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Four Digit Number : ");
scanf("%d",&x);
int first_two_digit, last_two_digit, reverse;
first_two_digit = x % 100;
last_two_digit = x / 100;
reverse = (last_two_digit/10) + ((last_two_digit %
10)*10);
y = (reverse * 100) + first_two_digit;
printf("Result =  %d",y);
}
```

Input :9561 **Output** : 5961.

17. Get a two-digit number from user and make the one's digit as 0, then print it.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = (x/10)*10;
printf("Result =  %d",y);
}
```

Input :59 **Output** : 50.

18. Get a two-digit number from user and make the ten's digit 1, then print it.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
y = 10 + (x%10);
printf("Result =  %d",y);
}
```

Input :82 **Output** : 12.

19. Get a three-digit number from user and make the one's digit as 2, then print it.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = ((x/10)*10) + 2;
printf("Result =  %d",y);
}
```

Input :695 **Output** : 692.

20. Get a three-digit number from user and make the ten's digit as 0, then print it.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
y = ((x/100)*100) + (x%10);
printf("Result =  %d",y);
}
```

Input :695 **Output** : 605.

21. Get a number from user and subtract 5 from that number if the number is odd, then print the result. Do not use "if".

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Number : ");
scanf("%d",&x);
y = x - ((x%2)*5);
printf("Result =  %d",y);
}
```

Input :695 **Output** : 690.
Input :566 **Output** : 566.

22. Get a number from user and subtract 5 from that number if the number's ten's position digit is odd, then print the result. Do not use "if".

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter a Number : ");
scanf("%d",&x);
y = x - ((((x/10)%10)%2)*5);
printf("Result =  %d",y);
}
```

Input :678 **Output** : 673.
Input :564 **Output** : 564.

23. Get a two digit number from user and subtract 5 from that number if the sum of the digits of the number is odd, then print the result. Do not use "if".

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
int sum = (x%10) + (x/10);
y = x - ((sum%2)*5);
printf("Result =  %d",y);
}
```

Input :95 **Output** : 95.
Input :72 **Output** : 67.

24. Get a three-digit number from user and subtract 5 from that number if one's digit number and 100's digit number are same, then print the result. Do not use "if".

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
int match = ((x/100) == (x%10));
y = x - (match*5);
printf("Result =  %d",y);
}
```

Input :595 **Output** : 590.
Input :372 **Output** : 372.

25. Get a four-digit number from user and subtract 5 from that number if ten's digit position and 100's digit position is same, then print the result. Do not use "if".

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter a Four Digit Number : ");
scanf("%d",&x);
int match = (((x/100)%10) == ((x/10)%10));
y = x - (match*5);
printf("Result =  %d",y);
}
```

Input :7595 **Output** : 7595.
Input :3772 **Output** : 3767.

26. Get a two-digit number from user. If the sum of the digits is 10 then print "Success", otherwise print "Failure".

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Two Digit Number : ");
scanf("%d",&x);
int sum = (x/10) + (x%10);
if (sum == 10) printf("Success");
else printf("Failure");
}
```

Input :56 **Output** : Failure.
Input :37 **Output** : Success.

27. Get a three-digit number from user. If the sum of the digits is 10 then print "Success", otherwise print "Failure".

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
int sum = (x/100) + ((x/10)%10) + (x%10);
if (sum == 10) printf("Success");
else printf("Failure");
}
```

Input :956 **Output** : Failure.
Input :127 **Output** : Success.

28. Get a three-digit number from user. If the sum of the one's digit and hundred's digit is less than 10, then print "Success", otherwise print "Failure".

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
int sum = (x/100) + (x%10);
if (sum < 10) printf("Success");
else printf("Failure");
}
```

Input :569 **Output** : Failure.
Input :316 **Output** : Success.

29. Get a four-digit number from user. If the sum of the ten's digit and hundred's digit is greater than 10, then print "Success", otherwise print "Failure".

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter a Four Digit Number : ");
scanf("%d",&x);
int sum = ((x/100)%10) + ((x/10)%10);
if (sum > 10) printf("Success");
else printf("Failure");
}
```

Input :7529 **Output** : Failure.
Input :9386 **Output** : Success.

30. Get a four-digit number from user. If the sum of the ten's digit and hundred's digit is equal to 10, and one of the digits is more than 7 then print "Success", otherwise print "Failure".

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter a Four Digit Number : ");
scanf("%d",&x);
int tens = ((x/10)%10);
int hundreds = ((x/100)%10);
int sum = hundreds + tens;
if (sum == 10){
   if((hundreds > 7) || (tens > 7))
   printf("Success");
   else printf("Failure");
}
else printf("Failure");
}
```

Input :4649 **Output** : Failure.
Input :9286 **Output** : Success.

31. Get a three-digit number from user. If the sum of the digits is less than 10, then print the sum, otherwise add the digits of the sum. If the sum of the digits is less than 10, then print the sum, otherwise add the digits of the sum, and print the sum. Note: The result should be always single digit only.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter a Three Digit Number : ");
scanf("%d",&x);
int ones = x%10;
int tens = ((x/10)%10);
int hundreds = ((x/100)%10);
int sum = hundreds + tens + ones;
if (sum < 10) printf("%d", sum);
else {
   x = sum;
   ones = x%10;
   tens = x/10;
   sum = tens + ones;
   if (sum < 10) printf("%d", sum);
   else {
      x = sum;
      ones = x%10;
      tens = x/10;
      sum = tens + ones;
      printf("%d", sum);
   }
}
}
```

Input: 123 **Output**: 6

Input: 149 **Output**: 5
(149:1+4+9 = 14: 1+4 = 5)

Input: 991 **Output**: 1
(991: 9+9+1 = 19: 1+9 = 10: 1+0 = 1)

32. Get two 2-digit numbers from user. If the sum of the numbers is less than 100, then print the sum, otherwise print the difference.

```
#include<stdio.h>
int main (){
int x,y;
printf("Enter 1st Two digit Number : ");
scanf("%d",&x);
printf("Enter 2nd Two digit Number : ");
scanf("%d",&y);
int sum = x + y;
if(sum < 100) printf("%d", sum);
else{
   int difference;
   if(x>y) difference = x - y;
   else difference = y - x;
   printf("%d", difference);
   }
}
```

Input :56, 78 **Output** : 22.
Input :14, 65 **Output** : 79.

33. Get two 2-digit numbers from user. Print the sum of digits of the biggest number.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter 1st Two digit Number : ");
scanf("%d",&x);
printf("Enter 2nd Two digit Number : ");
scanf("%d",&y);
int big_num, sum;

if(x>y) big_num = x;
else big_num = y;

sum = (big_num/10) + (big_num%10);
printf("%d", sum);
}
```

Input :56, 78 **Output** : 15.
Input :14, 65 **Output** : 11.

34. Get two 3-digit numbers from user. Print the difference between the one's digit and hundred's digit of the number whose ten's digit is bigger than the other number's ten's digit

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter 1st Three digit Number : ");
scanf("%d",&x);
printf("Enter 2nd Three digit Number : ");
scanf("%d",&y);
int number;
if(((x/10)%10) > ((y/10)%10)) number = x;
else number = y;
int ones = number%10;
int hundred = number/100;
if(ones > hundred) printf("%d", (ones - hundred));
else printf("%d", (hundred - ones));
}
```

Input :856, 978 **Output** : 1.
Input :128, 365 **Output** : 2.

35. Get two 3-digit numbers from user. Add the one's and hundred's digits of both the numbers. Print the sum of all the digits of the number whose sum of one's and hundred's digits is bigger.

```
#include<stdio.h>
int main ()
{
int x,y;
printf("Enter 1st Three digit Number : ");
scanf("%d",&x);
printf("Enter 2nd Three digit Number : ");
scanf("%d",&y);
int sum_x, sum_y, number;
sum_x = (x/100) + (x%10);
sum_y = (y/100) + (y%10);
if(sum_x > sum_y) number = x;
else number = y;
int sum = (number/100) + ((number/10)%10) +
(number%10);
printf("%d", sum);
}
```

Input :856, 978 **Output** : 24.
Input :128, 365 **Output** : 11.

Level - 2

01. Write a loop program to print 1 to 5 on one by one.

```
#include<stdio.h>
int main (){
int x;
x=1;
loop: if(x < 6)
{
printf("%d  ", x++);
goto loop;
}
}
```

Output : 1 2 3 4 5

02. Write a loop program to print 5 to 1 on one by one.

```
#include<stdio.h>
int main (){
int x;
x=5;
loop: if(x > 0)
{
printf("%d  ", x--);
goto loop;
}
}
```

Output : 5 4 3 2 1

03. Write a loop program to print sum of 1 to 5.

```
#include<stdio.h>
int main (){
int x = 1, sum = 0;
loop: if(x < 6)
{
sum += x++;
goto loop;
}
printf("%d", sum);
}
```

Output : 15

04. Write a loop program to print sum of 6 to 1.

```
#include<stdio.h>
int main (){
int x = 6, sum = 0;
loop: if(x > 0)
{
sum += x--;
goto loop;
}
printf("%d", sum);
}
```

Output : 21

05. Write a loop program to print odd numbers 1 to 9.

```
#include<stdio.h>
int main (){
int x = 1;
loop: if(x < 10)
{
if(x%2 == 1) printf("%d ", x);
x++;
goto loop;
}
}
```

Output : 1 3 5 7 9

06. Write a loop program to print the two-digit odd numbers, below 20.

```
#include<stdio.h>
int main (){
int x = 10;
loop: if(x < 20)
{
if(x%2 == 1) printf("%d ", x);
x++;
goto loop;
}
}
```

Output : 11 13 15 17 19

07. Write a loop program to print the two-digit odd numbers, who's sum of digits are 7.

```
#include<stdio.h>
int main (){
int x = 10, sum;
loop: if(x < 100){
if(x%2 == 1){
   sum = (x/10) + (x%10);
   if(sum == 7) printf("%d ", x);
}
x++;
goto loop;
} }
```

Output : 25 43 68

08. Write a loop program to print the two-digit even numbers, who's sum of digits are 6.

```
#include<stdio.h>
int main (){
int x = 10, sum;
loop: if(x < 100){
if(x%2 == 0){
   sum = (x/10) + (x%10);
   if(sum == 6) printf("%d ", x);
}
x++;
goto loop;
} }
```

Output : 24 42 60

09. Write a loop program to print the sum of two-digit numbers whose one's digit is 5.

```
#include<stdio.h>
int main (){
int x = 10, sum = 0;
loop: if(x < 100){
if(x%10 == 5){
   sum += x;
}
x++;
goto loop;
}
printf("%d", sum);
}
```

Output : 495 [15 + 25+ …….. + 85 + 95]

10. Write a loop program to print the sum of two-digit odd numbers, whose ten's digit is 7.

```
#include<stdio.h>
int main (){
int x = 10, sum = 0;
loop: if(x < 100){
   if(x%2 == 1){
      if(x/10 == 7){
         sum += x;
      }  }
x++;
goto loop;
}
printf("%d", sum);
}
```

Output : 375 [71 + 73 + 75 + 77 + 79]

11. Write a program to get a number from user and print the total number of digits in that number.

```
#include<stdio.h>
int main (){
   int x, count = 0;
   printf("Enter a Number");
   scanf("%d", &x);
   while(x){
      count++;
      x /= 10; }
   printf("%d", count);
}
```

Input : 123456 **Output** : 6.
Input : 675 **Output** : 3.

12. Write a program to get a number from user and print the sum of all digits.

```
#include<stdio.h>
int main (){
   int x, sum = 0;
   printf("Enter a Number");
   scanf("%d", &x);
   while(x){
      sum += (x%10);
      x /= 10;
   }
   printf("%d", sum);
}
```

Input : 123456 **Output** : 21.
Input : 675 **Output** : 18.

13. Write a program to get a number from user and print the reverse of that Number.

```
#include<stdio.h>
int main (){
   int x, reverse = 0;
   printf("Enter a Number");
   scanf("%d", &x);
   while(x){
      reverse = (reverse * 10) + (x%10);
      x /= 10;
   }
   printf("%d", reverse);
}
```

Input : 123456 **Output** : 654321.
Input : 675 **Output** : 576.

14. Write a program to get a number from user and interchange the first and last digits and print the result.

```
#include<stdio.h>
int main (){
   int x, y, first_digit, last_digit, mid_digits, mul = 1;
   printf("Enter a Number");
   scanf("%d", &x);
   y = x;
   while(y){ mul *= 10; y /= 10;}
   mul /= 10;
   first_digit = x%10;
   last_digit = x/mul;
   mid_digits = (x%mul) / 10;
   x = (first_digit*mul) + (mid_digits * 10) + last_digit;
   printf("%d", x);
}
```

Input : 123456 **Output** : 623451.
Input : 675 **Output** : 576.

15. Write a program to get a number from user and if the last digit of the number is even print the same number. If the last digit of the number is odd then subtract 1 from the last digit and print the number. (Note: Last digit - MSB).

```
#include<stdio.h>
int main (){
   int x, y, first_digit, last_digit, mid_digits, mul = 1;
   printf("Enter a Number");
   scanf("%d", &x);
   y = x;
   while(y){ mul *= 10; y /= 10;}
   mul /= 10;

   if((x/mul)%2 == 1){
      y = (x/mul) - 1;
      y *= mul;
      x = y + (x%mul);
   }
   printf("%d", x);
}
```

Input : 123456 **Output** : 23456.
Input : 675 **Output** : 675.
Input : 96584 **Output** : 86584.

16. Write a program get number from user print whether that number is prime or not.

```
#include<stdio.h>
int main ()
{
int x;
int prime = 1;
printf("Enter a Number: ");
scanf("%d",&x);

if(x == 1) prime = 0;

for(int i=2; i<x; i++){
   if(x%i == 0){prime = 0; break;}
}

if(prime) printf("Prime");
else printf("Not Prime");
}
```

Input : 1 **Output** : Not Prime.
Input : 5 **Output** : Prime.
Input : 10 **Output** : Not Prime.

17. Write a program to get a number from user, print whether that number is prime, and sum of digit is equal to 14.

```
#include<stdio.h>

int is_prime(int number){
   int prime = 1;
   if((number == 1) || (number == 0)) prime = 0;
   for(int i=2; i<number; i++){
      if(number%i == 0){prime = 0; break;}
   }
   return prime;
}


int is_sum_14(int number){
   int digit[10] = {0}, i = 0, sum = 0;
   while(number){
      digit[i++] = number%10; number /= 10;
   }
   i = 0;
   while(digit[i]){
      sum += digit[i++];
   }
   if(sum == 14) return 1;
   else return 0;
}


int main ()
{
int x, prime, fourteen;
printf("Enter a Number: ");
scanf("%d",&x);

prime = is_prime(x);
fourteen = is_sum_14(x);

   if(prime && fourteen)  printf("Prime & sum of the digits are 14");
else if(!prime && fourteen) printf("Not Ptime But sum of the digits are 14");
else if(prime && !fourteen) printf("Prime but sum of the digits are not 14");
else printf("Not Prime and sum of the digits are not 14");
}
```

Input : 59 **Output** : Prime & sum is 14
Input : 77 **Output** : Not Prime but sum is 14
Input : 13 **Output** : Not Prime & sum is not 14

18. Write a program to get number from user, print whether that number's first two digits (ten's digits and one's digit) is prime.

```
#include<stdio.h>

int is_prime(int number){
   int prime = 1;
   if((number == 1) || (number == 0)) prime = 0;
   for(int i=2; i<number; i++){
      if(number%i == 0){prime = 0; break;}
   }
   return prime;
}


int main ()
{
int x;
printf("Enter a Number: ");
scanf("%d",&x);

x %= 100;

if(is_prime(x)) printf("Prime");
else printf("Not Prime");
}
```

Input : 359 **Output** : Prime.
Input : 3577 **Output** : Not Prime.

19. Write a program to get a 4-digit number from user, print whether that number's middle two digits (hundred's digit and ten's digit) is prime.

```
#include<stdio.h>

int is_prime(int number){
   int prime = 1;
   if((number == 1) || (number == 0)) prime = 0;
   for(int i=2; i<number; i++){
      if(number%i == 0){prime = 0; break;}
   }
   return prime;
}

int main (){
int x;
printf("Enter a Four digit Number: ");
scanf("%d",&x);

x = (x%1000)/10;
if(is_prime(x)) printf("Prime");
else printf("Not Prime");
}
```

Input : 3537 **Output** : Prime.
Input : 6359 **Output** : Not Prime.

20. Write a program print total number of single digit Prime numbers.

```
#include<stdio.h>

int is_prime(int number){
   int prime = 1;
   if((number == 1) || (number == 0)) prime = 0;
   for(int i=2; i<number; i++){
      if(number%i == 0){prime = 0; break;}
   }
   return prime;
}


int main ()
{
int x, count = 0;

for(int i=0; i<10; i++){
   if(is_prime(i)) count++;
 }
 printf("%d", count);
}
```

Output : 4.

21. Write a program get number from user print the total number digits which are odd in the number.

```
#include<stdio.h>
int main ()
{
int x, count = 0, digit;
printf("Enter a Number: ");
scanf("%d",&x);

while(x){
   digit = x%10;
   if(digit%2 == 1) count++;
   x /= 10;
}
printf("%d", count);
}
```

Input : 12345678 **Output** : 4.
Input : 987531 **Output** : 5.

22. Write a program get number from user print the total number of two-digit odd numbers in the number.

```
#include<stdio.h>
int main ()
{
int x, count = 0, digit;
printf("Enter a Number: ");
scanf("%d",&x);

while(x > 9){
   digit = x%100;
   if(digit%2 == 1) count++;
   x /= 10;
}
printf("%d", count);
}
```

Input : 12345678 **Output** : 3.
Input : 987531 **Output** : 4.

23. Write a program get number from user print the total number of single-digit perfect square numbers in the number.

```
#include<stdio.h>
int main ()
{
int x, count = 0, digit;
printf("Enter a Number: ");
scanf("%d",&x);

while(x){
   digit = x%10;
   for(int i=1; i<10; i++){
      if((i*i) == digit){ count++; break;}
   }
   x /= 10;
}
printf("%d", count);
}
```

Input : 123456789 **Output** : 3.
Input : 987531 **Output** : 2.

24. Write a program get number from user print the total number of two-digit perfect square numbers in the number.

```
#include<stdio.h>
int main ()
{
int x, count = 0, digit;
printf("Enter a Number: ");
scanf("%d",&x);

while(x > 9){
   digit = x%100;
   for(int i=1; i<100; i++){
      if((i*i) == digit){ count++; break;}
   }
   x /= 10;
}
printf("%d", count);
}
```

Input : 163496481 **Output** : 4.
Input : 364925 **Output** : 4.

25. Write a program get number from user print the total number of single-digit prime numbers in the number.

```
#include<stdio.h>
int is_prime(int number){
   int prime = 1;
   if((number == 1) || (number == 0)) prime = 0;
   for(int i=2; i<number; i++){
      if(number%i == 0){prime = 0; break;}
   }
   return prime;
}

int main (){
int x, count = 0, digit;
printf("Enter a Number: ");
scanf("%d",&x);
while(x){
   digit = x%10;
   if(is_prime(digit)) count++;
   x /= 10;
}
printf("%d", count);
}
```

Input : 163496481 **Output** : 1.
Input : 364925 **Output** : 3.

26. Write a program to print biggest 4-digit number which is divisible by 7 and 9.

```
#include<stdio.h>
int main ()
{
   int i;
   for(i=9999; i>999; i--){
      if((i%7 == 0) && (i%9 == 0)) break;
   }
printf("%d", i);
}
```

Output : 9954.

27. Write a program to print the total count of numbers which are less than 100000 and whose sum of digits is 14.

```
#include<stdio.h>
int main ()
{
   int i, digit, count = 0, sum = 0, copy;
   for(i=99999; i>0; i--){
      copy = i;
      for(int j=0; j<6; j++){
         digit = copy%10;
         copy /= 10;
         sum = sum + digit;
      }
      if(sum == 14){count++; printf("%d\n", i);}
      sum = 0;
   }
  printf("\n\n Count = %d", count);
}
```

Output : 19004, 18500, ……… , 68, 59
Count = 2710

28. Write a program to get two numbers from user and print the LCM of those numbers.

```
#include<stdio.h>
int main (){
 int x, y;
 printf("Enter the First Number: ");
 scanf("%d", &x);
 printf("Enter the Second Number: ");
 scanf("%d", &y);

 int min, max;
 if(x>y) min = x;
 else min = y;
 max = x*y;

 for(int i=min; i<=max; i++){
  if((i%x == 0) && (i%y == 0)){
     printf("LCM = %d", i);
     break;
  } } }
```

Input : 4, 24 **Output** : 24.
Input : 5, 8 **Output** : 40.
Input : 24, 64 **Output** : 192.

29. Write a program to get three numbers from user and print the LCM of those numbers.

```
#include<stdio.h>
int main (){
 int x, y, z;
 printf("Enter the First Number: ");
 scanf("%d", &x);
 printf("Enter the Second Number: ");
 scanf("%d", &y);
 printf("Enter the Third Number: ");
 scanf("%d", &z);

 int min, max;
 if(x>y){ if(x>z) min = x; else min = z;}
 else { if(y>z) min = y; else min = z;}
 max = x*y*z;

 for(int i=min; i<=max; i++){
   if((i%x == 0) && (i%y == 0) && (i%z == 0)){
      printf("LCM = %d", i);
      break;
   } } }
```

Input : 2, 4, 8 **Output** : 8.
Input : 3, 6, 9 **Output** : 18.
Input : 15, 35, 5 **Output** : 105.

30. Write a program to get two numbers from user and print the HCF of those numbers.

```
#include<stdio.h>
int main ()
{
 int x, y, max;
 printf("Enter the First Number: ");
 scanf("%d", &x);
 printf("Enter the Second Number: ");
 scanf("%d", &y);

 if(x>y) max = y;
 else max = x;

 for(int i=max; i>0; i--){
  if((x%i == 0) && (y%i == 0)){
    printf("HCF = %d", i);
    break;
  }
 }
}
```

Input : 8, 20 **Output** : 4.
Input : 55, 61 **Output** : 1.
Input : 3524, 6122 **Output** : 2.

Level - 3

01. Get a number from user and add 2 to that number and print the result. Write your code inside the function. Do not Change the Code.

```
#include <stdio.h>

int function(int no);
int main(){
int number1, number2;
printf("Enter a Number: ");
scanf("%d",&number1);
number2 = function(number1);
printf("%d",number2);
return 0;
}

int function(int no1){
int no2;
no2 = no1 + 2;
return no2;
}
```

Input : 44 **Output** : 46.

02. Get a number from user and subtract 5 to that number and print the result. Write your code inside the function. Do not Change the format.

```
#include <stdio.h>

int function(int no);
int main(){
int number1, number2;
printf("Enter a Number: ");
scanf("%d",&number1);
number2 = function(number1);
printf("%d",number2);
return 0;
}
int function(int no1){
int no2;
no2 = no1 - 5;
return no2;
}
```

Input : 44 **Output** : 39.

03. Get a number from user and Check whether the sum of digits is 14 and print the result. Write your code inside the function. Do not Change the format.

```
#include <stdio.h>

int function(int no);
int main(){
int number1, number2;
printf("Enter a Number: ");
scanf("%d",&number1);
number2 = function(number1);
if(number2 == 14) printf("Sum is 14");
else printf("Sum is not 14");
return 0;
}

int function(int no1){
int sum = 0;
while(no1){
 sum += no1%10;
 no1 /= 10;
}
return sum;
}
```

Input : 45865 **Output** : Sum is not 14.
Input : 45865 **Output** : Sum is not 14.

04. Get a number from user and Check Prime or Not and print the result. Write your code inside the function. Do not Change the format.

```
#include <stdio.h>

int function(int no);
int main(){
int number1, number2;
printf("Enter a Number: ");
scanf("%d",&number1);
number2 = function(number1);
if(number2) printf("Prime");
else printf("Not Prime");
return 0;
}
int function(int no1){
   if((no1 == 0) || (no1 == 1)) return 0;
int prime = 1;
for(int i=2; i<no1;i++){
   if((no1%i) == 0){
      prime = 0;
      break;
   } }
return prime;
}
```

Input : 2 **Output** : Prime.
Input : 9 **Output** : Not Prime.

05. Get a number from user and count the number of zeros in that number and print. Write your code inside the function. Do not Change the format.

```
#include <stdio.h>
int find_no_of_zeros(int no);
int main(){
int number,result;
printf("Enter a number");
scanf("%d",&number);
result = find_no_of_zeros (number);
printf("%d",result);
return 0;
}
int find_no_of_zeros (int no){
int count = 0, rem;
while(no){
   rem = no%10;
   if(rem == 0) count++;
   no /= 10;
}
return count;
}
```

Input : 40805 **Output** : 2.
Input : 45865 **Output** : 0.

06. Get a number from user and reverse that number and print. Write your code inside the function. Do not Change the format.

```
#include <stdio.h>
int reverse_number(int no);
int main(){
int number,result;
printf("Enter a number");
scanf("%d",&number);
result = reverse_number (number);
printf("%d",result);
return 0;
}

int reverse_number (int no){
int rev = 0;
while(no){
   rev = (rev*10) + no%10;
   no /= 10;
}
return rev;
}
```

Input : 42805 **Output** : 50824.
Input : 45865 **Output** : 56865.

07. Get two numbers from user and compare the numbers. If same print "Same" otherwise print "Not Same". Write your code inside the function. Do not Change the format.

```
#include <stdio.h>
int comp_number(int no1, int no2);
int main(){
int num1, num2,result;
printf("Enter 1st number: ");
scanf("%d",&num1);
printf("Enter 2nd number: ");
scanf("%d",&num2);
result = comp_number (num1, num2);
if(result) printf("Same");
else printf("Not same");
return 0;
}

int comp_number (int no1, int no2){
return (no1 == no2);
}
```

Input : 45, 45 **Output** : Same.
Input : 8, 9 **Output** : Not same.

08. Get a number from user and check whether the digits are in ascending order.

```
#include <stdio.h>
int check_ascending(int no);
int main(){
int number,result;
printf("Enter a number: ");
scanf("%d",&number);
result = check_ascending (number);
if(result) printf("Ascending");
else printf("Not Ascending");
return 0;
}

int check_ascending (int no){
int ascending = 1, last_digit;
while(no){
   last_digit = no%10;
   no /= 10;
   if(last_digit < no%10){
      ascending = 0;
      break;
   } }
 return ascending;
}
```

Input : 123456 **Output** : Ascending.
Input : 123454 **Output** : Not Ascending.

09. Get a two-digit number from user swap the digits.

```
#include <stdio.h>

int swap_digit(int no);
int main(){
int number,result;
printf("Enter a two digit number: ");
scanf("%d",&number);
result = swap_digit (number);
printf("%d", result);
return 0;
}

int swap_digit (int no){
int result;
result = ((no%10)*10) + (no/10);
return result;
}
```

Input : 45 **Output** : 54.
Input : 29 **Output** : 92

10. Get a number from user, find the number of digits and print the same.

```
#include <stdio.h>

int count_digits(int no);
int main(){
int number,result;
printf("Enter a number: ");
scanf("%d",&number);
result = count_digits (number);
printf("%d", result);
return 0;
}

int count_digits (int no){
int count = 0;
while(no){
   count++;
   no /= 10;
}
return count;
}
```

Input : 45692 **Output** : 5.
Input : 804 **Output** : 3

Level - 4

01. Get a Two-digit number from user and print the digit in "Ones" position.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a TWO-digit Number: ");
scanf ("%d", &x);
y = x%10;
printf ("The digit in Ones position is %d\n",y);
}
```

Input : 45 **Output** : 5.
Input : 29 **Output** : 9.

02. Get a Two digit number from user and print the digit in "Tens" position.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a TWO-digit Number: ");
scanf ("%d", &x);
y = x/10;
printf ("The digit in Tens position is %d\n",y);
}
```

Input : 45 **Output** : 4.
Input : 29 **Output** : 2.

03. Get a Three digit number from user and print the digit in "Ones" position.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a THREE-digit Number: ");
scanf ("%d", &x);
y = x%10;
printf ("The digit in Ones position is %d\n",y);
}
```

Input : 745 **Output** : 5.
Input : 291 **Output** : 1.

04. Get a Three digit number from user and print the digit in "Tens" Position.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a THREE-digit Number: ");
scanf ("%d", &x);
y = (x/10)%10;
printf ("The digit in Tens position is %d\n",y);
}
```

Input : 745 **Output** : 4.
Input : 291 **Output** : 9.

05. Get a Three digit number from user and print the digit in "Hundreds" Position.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a THREE-digit Number: ");
scanf ("%d", &x);
y = x/100;
printf ("The digit in Hundreds position is %d\n",y);
}
```

Input : 745 **Output** : 7.
Input : 291 **Output** : 2.

06. Get a Two digit number from the user and print the reverse of it.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a TWO-digit Number: ");
scanf ("%d", &x);
y = ((x%10)*10) + (x/10);
printf ("The reverse of the number is %d\n",y);
}
```

Input : 74 **Output** : 47.
Input : 21 **Output** : 12.

07. Get a Three digit number from the user and print the reverse of it.

```
#include<stdio.h>

int main (){
int x, y;
printf ("Enter a THREE-digit Number: ");
scanf ("%d", &x);
y = ((x%10)*100) + (((x/10)%10)*10) + (x/100);
printf ("The reverse of the number is %d\n",y);
}
```

Input : 745 **Output** : 547.
Input : 291 **Output** : 192.

08. Get a Four digit number from the user and print the reverse of it.

```
#include<stdio.h>
int main (){
int x, reverse = 0;
printf ("Enter a FOUR-digit Number: ");
scanf ("%d", &x);
while(x){
   reverse = (reverse * 10) + (x%10);
   x /= 10;
}
printf ("The reverse of the number is %d\n",reverse);
}
```

Input : 7245 **Output** : 5427.
Input : 2981 **Output** : 1892.

09. Get a Two digit number from the user and print the sum of all digits.

```
#include<stdio.h>
int main (){
int x, sum = 0;
printf ("Enter a TWO-digit Number: ");
scanf ("%d", &x);
while(x){
   sum = sum + (x%10);
   x /= 10;
}
printf ("The sum of all digit is %d\n",sum);
}
```

Input : 75 **Output** : 12.
Input : 91 **Output** : 10.

10. Get a Three digit number from the user and print the sum of all digits.

```
#include<stdio.h>
int main (){
int x, sum = 0;
printf ("Enter a THREE-digit Number: ");
scanf ("%d", &x);
while(x){
   sum = sum + (x%10);
   x /= 10;
}
printf ("The sum of all digit is %d\n",sum);
}
```

Input : 745 **Output** : 16.
Input : 291 **Output** : 12.

11. Get a Four digit number from the user and print the sum of all digits.

```
#include<stdio.h>

int main (){
int x, sum = 0;
printf ("Enter a FOUR-digit Number: ");
scanf ("%d", &x);
while(x){
   sum = sum + (x%10);
   x /= 10;
}
printf ("The sum of all digit is %d\n",sum);
}
```

Input : 7245 **Output** : 18.
Input : 2981 **Output** : 20.

12. Get a number from the user and print the reverse of it.

```
#include<stdio.h>

int main (){
int x, reverse = 0;
printf ("Enter a Number: ");
scanf ("%d", &x);
while(x){
   reverse = (reverse * 10) + (x%10);
   x /= 10;
}
printf ("The Reverse Number is %d\n",reverse);
}
```

Input : 123456 **Output** : 654321.
Input : 458561 **Output** : 165854.

13. Get a number from the user and print the sum of all digits.

```
#include<stdio.h>
int main (){
int x, sum = 0;
printf ("Enter a Number: ");
scanf ("%d", &x);
while(x){
   sum += x%10;
   x /= 10;
}
printf ("The Sum of all digits is %d\n",sum);
}
```

Input : 1234567 **Output** : 28.
Input : 2981 **Output** : 20.

14. Write a program to print the total number of single digit odd Numbers.

```
#include<stdio.h>

int main (){
int x, count = 0;
for(int i=0; i<10; i++){
   if(i%2 == 1) count++;
}
printf ("Result is %d\n",count);
}
```

Output : Result is 5.

15. Write a program to print the total number of TWO digit odd numbers.

```
#include<stdio.h>

int main (){
int x, count = 0;
for(int i=10; i<100; i++){
   if(i%2 == 1) count++;
}
printf ("Result is %d\n",count);
}
```

Output : Result is 45.

16. write a program to print the total number of THREE digit odd numbers.

```
#include<stdio.h>

int main (){
int x, count = 0;
for(int i=100; i<1000; i++){
   if(i%2 == 1) count++;
}
printf ("Result is %d\n",count);
}
```

Output : Result is 450.

17. Write a program to print the sum of all single digit odd numbers.

```
#include<stdio.h>

int main (){
int x, sum = 0;
for(int i=0; i<10; i++){
   if(i%2 == 1) sum += i;
}
printf ("Result is %d\n",sum);
}
```

Output : Result is 25.

18. Write a program to print the sum of all TWO digit odd numbers.

```
#include<stdio.h>

int main (){
int x, sum = 0;
for(int i=10; i<100; i++){
   if(i%2 == 1) sum += i;
}
printf ("Result is %d\n",sum);
}
```

Output : Result is 2475.

19. Write a program to print the sum of all THREE digit odd numbers.

```
#include<stdio.h>

int main (){
int x, sum = 0;
for(int i=100; i<1000; i++){
   if(i%2 == 1) sum += i;
}
printf ("Result is %d\n",sum);
}
```

Output : Result is 247500.

20. Write a program to print the total number of single digit Prime numbers Assume 0 & 1 are not Prime.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, count=0, prime;
for(int i=0; i<10; i++){
  prime = is_prime(i);
  if(prime) count++;
}
printf ("%d\n",count);
}
```

Output : 4

21. Write a program to print the total number of TWO digit Prime numbers.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, count=0, prime;
for(int i=10; i<100; i++){
  prime = is_prime(i);
  if(prime) count++;
}
printf ("%d\n",count);
}
```

Output : 21

22. Write a program to print total number of THREE digit Prime numbers.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, count=0, prime;
for(int i=100; i<1000; i++){
  prime = is_prime(i);
  if(prime) count++;
}
printf ("%d\n",count);
}
```

Output : 143

23. Write a program to print the sum of single digit Prime numbers.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, sum=0, prime;
for(int i=0; i<10; i++){
  prime = is_prime(i);
  if(prime) sum += i;
}
printf ("%d\n",sum);
}
```

Output : 17

24. Program to print the sum of all TWO digit Prime numbers.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, sum=0, prime;
for(int i=10; i<100; i++){
  prime = is_prime(i);
  if(prime) sum += i;
}
printf ("%d\n",sum);
}
```

Output : 1043

25. Program to Print the sum of all THREE digit Prime numbers.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, sum=0, prime;
for(int i=100; i<1000; i++){
  prime = is_prime(i);
  if(prime) sum += i;
}
printf ("%d\n",sum);
}
```

Output : 75067

26. Print the smallest Three digit prime number.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int x, i, prime;
for(i=100; i<1000; i++){
  prime = is_prime(i);
  if(prime) break;
}
printf ("%d\n",i);
}
```

Output : 101

27. Print the largest Three digit prime number.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int i, prime;
for(i=999; i>=100; i--){
  prime = is_prime(i);
  if(prime) break;
}
printf ("%d\n",i);
}
```

Output : 997

28. Print the Smallest Four digit prime number.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int i, prime;
for(i=1000; i<10000; i++){
  prime = is_prime(i);
  if(prime) break;
}
printf ("%d\n",i);
}
```

Output : 1009

29. Print the Largest Four digit prime number.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int i, prime;
for(i=9999; i>=1000; i--){
  prime = is_prime(i);
  if(prime) break;
}
printf ("%d\n",i);
}
```

Output : 9973

30. Print the Largest eight-digit prime number.

```
#include<stdio.h>

int is_prime(int num){
   int prime = 1;
   if((num == 1) || (num == 0)) return 0;
   for(int i=2; i<num; i++){
      if((num%i) == 0){ prime = 0; break;}
   }
   return prime;
}

int main (){
int i, prime;
for(i=99999999; i>=10000000; i--){
  prime = is_prime(i);
  if(prime) break;
}
printf ("%d\n",i);
}
```

Output : 99999989

31. Print the number of zeroes you encounter between the numbers 0 to 1000.

```
#include<stdio.h>

int count_zeros(int num){
 int count = 0, rem;
   while(num){
     rem = num % 10;
     if(rem == 0) count++;
     num /= 10;
   }
   return count;
}

int main (){
int result, count = 0;
for(int i=0; i<=1000; i++){
 result = count_zeros(i);
 count = count + result;
}
printf ("%d\n",count);
}
```

Output : 192

32. Total number of prime numbers below 1,000,000 have the sum of their digits equal to 14? Example: 59, 5 + 9 = 14.

```
#include<stdio.h>

int is_prime(int num){
    int prime = 1;
    if((num == 1) || (num == 0)) return 0;
    for(int i=2; i<num; i++){
        if((num%i) == 0){ prime = 0; break;}
    }
    return prime;
}

int is_sum_14(int num){
  int rem, sum = 0;
  while(num){
    rem = num % 10;
    sum += rem;
    num /= 10;
  }
  return sum;
}


int main (){
int prime, count = 0;
for(int i=0; i<=1000000; i++){
  prime = is_prime(i);
  if(prime){
    if((is_sum_14(i)) == 14) count++;
  }
}
printf ("No. of prime and Sum 14 is: %d\n",count);
}
```

Output : No. of prime and Sum 14 is: 1218

33. Print the total number of non-decreasing numbers from 1000 to 9999.Non decreasing numbers have individual digits that do not have a decreasing order from left to right. (For e.g.: 1234 is a non-decreasing number where as 2134 is not).

```
#include<stdio.h>
int is_non_decreasing(int num){
  int rem, result = 1;
    while(num){
      rem = num % 10;
      num /= 10;
      if(rem < (num % 10)){ result = 0; break;}
    }
    return result;
}

int main (){
int result, count = 0;
for(int i=1000; i<=9999; i++){
  result = is_non_decreasing(i);
  if(result){ count++; printf("%d\n", i);}
  }
printf ("Total non-decreasing number: %d\n",count);
}
```

Output : Total non-decreasing number: 495

34. Print the total number of all Palindrome numbers less than 100000. Example: 101, 12321, 656, 99899, 11511.

```
#include<stdio.h>

int is_Palindrome(int num){
  int rev = 0, num_copy = num;
  while(num){
    rev = (rev * 10) + (num % 10);
    num /= 10;
  }
  if(num_copy == rev) return 1;
  else return 0;
}

int main (){
int result, count = 0;
for(int i=0; i<=100000; i++){
  result = is_Palindrome(i);
  if(result){ count++; printf("%d\n", i);}
  }
printf ("Total Palindrome number: %d\n",count);
}
```

Output : Total Palindrome number: 1099

35. Get 2 numbers from user and find the LCM of them.

```
#include<stdio.h>

int LCM(int num1, int num2){
  int min, max, i;
  if(num1 > num2) min = num1;
  else min = num2;
  max = num1 * num2;
  for(i=min; i<=max; i++){
    if(((i%num1) == 0) && ((i%num2) == 0)) break;
  }
  return i;
}

int main (){
int x, y, result;
printf("Enter 1st Number: ");
scanf("%d", &x);
printf("Enter 2nd Number: ");
scanf("%d", &y);
result = LCM(x,y);
printf("%d", result);
}
```

Input : 12, 15 **Output** : 60

36. Get a character and print its ASCII Value.

```
#include<stdio.h>

int main (){
char x;
printf("Enter a Character: ");
scanf("%c", &x);
printf("%d", x);
}
```

Input : A **Output** : 65
Input : a **Output** : 97

37. Get a number and print its ASCII Value.

```
#include<stdio.h>

int main (){
char x;
printf("Enter a Number: ");
scanf("%c", &x);
printf("%d", x);
}
```

Input : 0 **Output** : 48
Input : 9 **Output** : 57

38. Get a String and print the same.

```
#include<stdio.h>

int main (){
char x[20];
printf("Enter a String: ");
scanf("%s", &x);
printf("%s", x);
}
```

Input : ECEN **Output** : ECEN
Input : Dharani **Output** : Dharani

39. Get a number as String and print the integer value of the string.

```
#include<stdio.h>

int main (){
char x[20];
int i = 0;
printf("Enter a Number: ");
scanf("%s", &x);
while(x[i]){
   printf("The Integer value of %c is %d\n", x[i], x[i]);
   i++;
   }
}
```

Input : 1234 **Output** : The Integer value of 1 is 49
The Integer value of 2 is 50
The Integer value of 3 is 51
The Integer value of 4 is 52

40. Get an integer and print it as string.

```
#include<stdio.h>

int main (){
int number;
printf("Enter a Number: ");
scanf("%d", &number);
printf("%c", number);
}
```

Input : 65 **Output** : A
Input : 48 **Output** : 0

41. Get an integer and print each digit as character. Print one character on one line.

```
#include<stdio.h>
int main (){
int number, rev_num = 0;
printf("Enter a Number: ");
scanf("%d", &number);

while(number){
   rev_num = (rev_num * 10) + (number % 10);
   number /= 10;
}
while(rev_num){
   printf("%c\n", ((rev_num % 10) + '0'));
   rev_num /= 10;
} }
```

Input : 1234 **Output** : 1
2
3
4

42. Get a string and find the length of the string.

```
#include<stdio.h>

int main (){
char string[20];
printf("Enter a string: ");
scanf("%s", string);
int i = 0, length = 0;
while(string[i]){
   length++;
   i++;
}
printf("%d", length);
}
```

Input : Dharani **Output** : 7
Input : ECEN **Output** : 4

43. Get a String of numbers up to 50 digits and validate the number.

```
#include<stdio.h>

int main (){
char number[50];
printf("Enter a Number: ");
scanf("%s", number);
int i = 0, valid = 1;
while(number[i]){
   if(((number[i]) < '0') || ((number[i]) > '9')){
      valid = 0;
      break;
   }
   i++;
  }
  if(valid) printf("Valid Number");
  else printf("Not Valid Number");
}
```

Input : 123456 **Output** : Valid Number
Input : 1245g65 **Output** : Not Valid Number

44. Get a string of numbers up to 50 digits and remove all leading zeros.

```
#include<stdio.h>

int main (){
   char number[50];
   printf("Enter a Number: ");
   scanf("%s", number);
   int i = 0;
   while(number[i]){
      if((number[i]) != '0') break;
      i++;
   }
   printf("%s", &number[i]);
}
```

Input : 123456 **Output** : 123456
Input : 00000123 **Output** : 123

45. Get a number up to 50 digits and reverse it.

```
#include<stdio.h>

int main (){
   char number[50];
   char number2[50];
   printf("Enter a Number: ");
   scanf("%s", number);

   int length = 0, length2 = 0;
   while(number[length]) length++;
   length--;

   while(length >= 0){
      number2[length2++] = number[length--];
   }
   number2[length2] = 0;
   printf("%s", number2);
}
```

Input : 123456 **Output** : 654321
Input : ABCDEFG **Output** : GFEDCBA

46. Get a number string up to 50 digits and convert it to integer array.

```
#include<stdio.h>

int main (){
   char number[50];
   int number2[50];
   printf("Enter a Number: ");
   scanf("%s", number);
   int length = 0, i;
   while(number[length]) length++;
   length--;
  for(i=0; i<=length; i++){
   number2[i] = (number[i] - '0');
   printf("%d", number2[i]);
  }
}
```

Input : 123456 **Output** : 123456

47. Add two integer arrays of up to 50 digits and store the result in a 51 digits array. (page – 1)

```
#include<stdio.h>

void add_n_print_result(int *no1, int *no2){
   int num1[50], num2[50], ans[52];
   int rev_num1[50], rev_num2[50];
   int length1=0, length2=0, i, j;

// store the numbers into num1 array
   i=0;
   while(*no1 != 0) num1[i++] = *no1++;
   num1[i] = 0;
   length1 = i-1;

// store the numbers into num2 array
   i=0;
   while(*no2 != 0) num2[i++] = *no2++;
   num2[i] = 0;
   length2 = i-1;

// reverse number 1
   for(i=0, j=length1; j >= 0; i++, j--) rev_num1[i] =
num1[j];
   rev_num1[i] = 0;

// reverse number 2
   for(i=0, j=length2; j >= 0; i++, j--) rev_num2[i] =
num2[j];
   rev_num2[i] = 0;

   int add_length, carry=0;
   if(length1 > length2) add_length = length2;
   else add_length = length1;

// adding the two numbers upto minimum length
   for(i=0; i<=add_length; i++){
      ans[i] = rev_num1[i] + rev_num2[i] + carry;
      carry = ans[i] / 10;
      ans[i] %= 10;
   }

// adding the rest numbers with 0
   if(length1 > length2)
      while(rev_num1[i] != 0){
         ans[i] = rev_num1[i] + carry;
         carry = 0;
         i++;
      }
```

(page – 2)

```
else
      while(rev_num2[i] != 0){
         ans[i] = rev_num2[i] + carry;
         carry = 0;
         i++;
      }
   ans[i++] = carry;  // for the secure of carry if occurs
   ans[i] = 0;

// skip the leading zeros
   while(ans[i] == 0) i--;

// printing the answer
   for(; i>=0; i--)
   printf("%d", ans[i]);
}

int main (){
   int number1[50]={9,6,7,5,7,2,3,4};
   int number2[50]={7,4,8,5,4,6,8};
   int result[52];
   add_n_print_result(number1, number2);
}
```

Output : 104242702

48. Adjust the carry in an integer array. (i.e. convert the 2 digit number into single digit and add the carry to the next number).

```
#include<stdio.h>

int main (){
   int number[10]={6,12,3,15,7}, rev_num[10];
   int result[12];

   int length=0, i=0, carry=0;
   while(number[length] != 0) length++;
   length--;

   while(length >= 0){
      rev_num[i++] = number[length--];
   }
   rev_num[i] = 0;

   for(i=0; rev_num[i] != 0; i++){
      result[i] = (rev_num[i] % 10) + carry;
      carry = rev_num[i] / 10;
   }
   result[i++] = carry;
   result[i] = 0;

   while(result[i] == 0) i--; // skip the leading zeros

   for(; i>=0; i--) printf("%d", result[i]);
}
```

Output : 7 2 4 5 7

49. Convert an integer array of up to 50 digits to character array and print using print using "printf("%s", ...);"

```
#include<stdio.h>

int main (){
   int number[20]={6,2,3,5,7,2,5,9,4}, i=0;
   char result[20];

   while(number[i] != 0){
      result[i] = number[i] + '0';
      i++;
   }
   result[i] = 0;

   printf("%s", result);
}
```

Output : 623572594

50. Get two numbers of up to 50 digits and perform addition and print the result. (page – 1)

```
#include<stdio.h>

void add_n_print_result(char *no1, char *no2){
   char num1[50], num2[50], rev_ans[52], ans[52];
   char rev_num1[50], rev_num2[50];
   int length1=0, length2=0, i, j;

// store the numbers into num1 array
   i=0;
   while(*no1 != 0) num1[i++] = *no1++;
   num1[i] = 0;
   length1 = i-1;

// store the numbers into num2 array
   i=0;
   while(*no2 != 0) num2[i++] = *no2++;
   num2[i] = 0;
   length2 = i-1;

// reverse number 1
   for(i=0, j=length1; j >= 0; i++, j--) rev_num1[i] =
num1[j];
   rev_num1[i] = 0;

// reverse number 2
   for(i=0, j=length2; j >= 0; i++, j--) rev_num2[i] =
num2[j];
   rev_num2[i] = 0;

   int add_length, carry=0;
   if(length1 > length2) add_length = length2;
   else add_length = length1;

// adding the two numbers upto minimum length
   for(i=0; i<=add_length; i++){
      rev_ans[i] = (rev_num1[i] - '0') + (rev_num2[i] - '0')
+ ((char)carry);
      carry = rev_ans[i] / 10;
      rev_ans[i] %= 10;
   }

// adding the rest numbers with 0
   if(length1 > length2)
      while(rev_num1[i] != 0){
         rev_ans[i] = (rev_num1[i] - '0') + ((char)carry);
         carry = 0;
         i++;
      }
```

(page – 2)

```
   else
      while(rev_num2[i] != 0){
         rev_ans[i] = (rev_num2[i] - '0') + ((char)carry);
         carry = 0;
         i++;
      }
   rev_ans[i++] = (char)carry;  // for the secure of carry
if occurs
   rev_ans[i] = 0;

   while(rev_ans[i] == 0) i--; // skip the zeros

   for(int j=0; i>=0; j++, i--) ans[j] = (rev_ans[i]) + '0';
   ans[--j] = 0;  // null character

   printf("%s", ans);
}

int main (){
   char number1[50];
   char number2[50];
   printf("Enter the 1st number: ");
   scanf("%s", number1);
   printf("Enter the 2nd Number: ");
   scanf("%s", number2);
   add_n_print_result(number1, number2);
}
```

Input : 4567, 789561 **Output** : 794128

51. Get a string and a character from the user and find all the positions where the character present and print it.

```
#include<stdio.h>

int main (){
   char string[20];
   char character[2]; // here, we requires only one
character but want to allocate 2 , without two, doesn't
work

   printf("Enter a string: ");
   scanf("%s", string);
   printf("Enter a character: ");
   scanf("%s", character);

   int i=0;
   while(string[i] != 0){
      if(string[i] == character[0])
      printf("%d, ", (i+1));
      i++;
   }
}
```

Input : string : ecenacademy   **Output** : 1, 3, 9,
           character: e

53. Get a string using gets function and count all the words in it.

```
#include<stdio.h>
int main (){
   char sentence[100];
   int count = 0;

   printf("Enter a Sentence: ");
   gets(sentence);
   char *ptr = sentence;

   while(*ptr != 0){
      if(*ptr == ' '){
         count++;
         while(*ptr == ' ') ptr++; // to skip the double
spaces
      }
      ptr++;
   }
   printf("%d", (count + 1));
}
```

Input : Welcome to ECEN Academy   **Output** : 4

52. Get a main string and sub string. Check the sub string in main string and print the position.

```
#include<stdio.h>

int main (){
   char main_string[20];
   char sub_string[10];
   int position = 1;

   printf("Enter a Main String: ");
   scanf("%s", main_string);
   printf("Enter a Sub String: ");
   scanf("%s", sub_string);

   char *main_ptr, *sub_ptr;
   main_ptr = main_string;
   sub_ptr = sub_string;

// Detecting the first character
   while(*sub_ptr != *main_ptr){ main_ptr++;
position++; }

// Checking whether the string is matched or not
   while(*sub_ptr != 0){
      if(*sub_ptr != *main_ptr){
         position = 0;
         break;
      }
      sub_ptr++;
      main_ptr++;
   }
   if(position) printf("%d", position);
   else printf("No Match");
}
```

Input : string : hellosurabee   **Output** : 6
           character: sura

54. Write a program to multiply up to two 50 digit numbers.

```
#include<stdio.h>

int main(){
  char num1[51];
  char num2[51];
  int revNum1[50];
  int revNum2[50];
  int ans[100]={0};

  printf("Enter 1st Number");
  scanf("%s", num1);
  printf("Enter 2nd Number");
  scanf("%s", num2);

  int i=0, j=0, l1, l2;
  while(num1[i]) i++; i--;  // length of the string1
  while(num2[j]) j++; j--;  // length of the string2
  l1=i;
  l2=j;

  for( i=l1, j=0; i>=0; i--, j++)
  revNum1[j]=num1[i]-48;

  for(i=l2, j=0; i>=0; i--, j++)
  revNum2[j]=num2[i]-48;

  for(i=0; i<=l1; i++){
    for(j=0; j<=l2; j++){
      ans[i+j] += revNum1[i]*revNum2[j];
    }
  }

  int carry=0;
  for(i=0; i<=(l1+l2);i++){
    carry=ans[i]/10;
    ans[i]%=10;
    ans[i+1]=ans[i+1]+carry;
  }

  i=(l1+l2);
  while(ans[i] == 0) i--;

  for(;i>=0; i--){
    printf("%d",ans[i]);
  }
}
```

Input : 25896, 25432 **Output** : 658587072

55. Write a Calculator program that will give the "Calc" prompt and always stay on this prompt. When a user types one of the following commands, the program will calculate and give the result. Typing "Exit" will exit from the Calculator program. This program accepts up to 50-digit numbers. Then, the division will give the quotient and remainder.

```
#include<stdio.h>               //     (page – 1)

void addition();
void subtraction();
void multiplication();
void division();


int main()
{
   char number[5];
   char exit[5]="exit";
loop:
 printf("\n\nCalc>\n");
 printf("a> Addition\n");
 printf("b> Subtraction\n");
 printf("c> Multiplication\n");
 printf("d> Division\n");
 printf("Enter your choice\n");

 scanf("%s",number);
 for(int i=0; i<=3; i++)
 {
  if(number[i]==exit[i])
  return 0;
 }
 switch(number[0])
 {
   case 97: printf("\nAddition\n");
           addition();
           break;
   case 98: printf("\nSubtraction\n");
           subtraction();
           break;
   case 99: printf("\nMultiplication\n");
           multiplication();
           break;
   case 100:printf("\nDivision\n");
           division();
           break;
   default: printf("invalid input");
  }
  goto loop;
}
```

```c
void addition()                    //      (page – 2)
{
 char num1[51];
  char num2[51];
  int revNum1[50]={0};
  int revNum2[50]={0};
  int ans[51]={0};

  printf("Enter 1st Number: ");
  scanf("%s", num1);
  printf("Enter 2nd Number: ");
  scanf("%s", num2);

  int i=0, j=0, l1, l2, max;
  while(num1[i]) i++; i--;  // length of the string1
  while(num2[j]) j++; j--;  // length of the string2
  l1=i;
  l2=j;

  if(l1>l2) max=l1;
  else max=l2;

  for( i=l1, j=0; i>=0; i--, j++)  // reverse numbe1
  revNum1[j]=num1[i]-48;

  for(i=l2, j=0; i>=0; i--, j++)  // reverse number2
  revNum2[j]=num2[i]-48;

   int carry=0;
  for(i=0; i<=max; i++)
  {
   ans[i]=revNum1[i] + revNum2[i] + carry;
   carry=ans[i]/10;
   ans[i]%=10;
  }
  ans[i]=carry;

  for(; i>=0; i--)
  {
    if(ans[i]>0)
    break;
  }

  printf("Answer is: ");
  for(;i>=0; i--)
  {
    printf("%d",ans[i]);
  }
}

void subtraction()
{
 char num1[51], num2[51];
  int intNum1[50]={0}, intNum2[50]={0};
  int ans[51];
  long long divisor;
```

```c
printf("Enter large Number: ");                    //      (page – 3)
 scanf("%s", num1);
 printf("Enter small Number: ");
 scanf("%s", num2);

 int i=0, j=0, len1, len2, length;
 while(num1[i]) i++; i--;  // length of the 1st string
 while(num2[j]) j++; j--;  // length of the 2nd string
 len1=i;
 len2=j;
 if(len1>len2) length=len1;
 else length=len2;

 for(i=0, j=len1; i<=len1; i++, j--)  // reverse 1st number
 intNum1[j]=num1[i]-48;

for(i=0, j=len2; i<=len2; i++, j--)  // reverse 2nd number
 intNum2[j]=num2[i]-48;

int borrow=0;
for(i=0; i<=length; i++)
{
   if(intNum1[i]>=intNum2[i])
   {
   ans[i]=intNum1[i] - (intNum2[i]+borrow);
   borrow=0;
   }

   else
   {
      intNum1[i]+=10;
      ans[i]=intNum1[i] - (intNum2[i]+borrow);
      borrow=1;
   }
}
for(i=length; i>=0; i--)  //eliminate the leading zeros
if(ans[i] !=0)
break;

printf("Answer is: ");
for(; i>=0; i--)
printf("%d",ans[i]);
}

void multiplication()
{
char num1[51];
 char num2[51];
 int revNum1[50];
 int revNum2[50];
 int ans[100]={0};

 printf("Enter 1st Number: ");
 scanf("%s", num1);
 printf("Enter 2nd Number: ");
 scanf("%s", num2);
```

```
 int i=0, j=0, l1, l2;                    //  (page – 4)
 while(num1[i]) i++; i--;  // length of the string1
 while(num2[j]) j++; j--;  // length of the string2
 l1=i;
 l2=j;

 for( i=l1, j=0; i>=0; i--, j++)
 revNum1[j]=num1[i]-48;

 for(i=l2, j=0; i>=0; i--, j++)
 revNum2[j]=num2[i]-48;

 for(i=0; i<=l1; i++)
 {
  for(j=0; j<=l2; j++)
  {
   ans[i+j] += revNum1[i]*revNum2[j];
  }
 }

 int carry=0;
 for(i=0; i<=(l1+l2);i++)
 {
  carry=ans[i]/10;
  ans[i]%=10;
  ans[i+1]=ans[i+1]+carry;
 }

 for(i=(l1+l2); i>=0; i--)
 {
  if(ans[i]!=0)
  break;
 }

 printf("Answer is: ");
 for(i=(i+1);i>=0; i--)
 {
  printf("%d",ans[i]);
 }
}

void division()
{
 char num[51];
 char ans[51];
 long long divisor;

 printf("Enter Divident Number: ");
 scanf("%s", num);
 printf("Enter Divisor Number: ");
 scanf("%llu", &divisor);

 int i=0, len;
 while(num[i]) i++; i--;  // length of the string
 len=i;

 for(i=0; i<=len; i++)  // string to integer conversion
 num[i]-=48;
```

```
 long long rem, temp;                 //    (page – 5)
 int k=0, flag=0;;

 temp=num[0];
 rem=temp;
 for(i=1; i<=(len+1); i++)
 {
  if(temp>=divisor)
  {
     ans[k++]=temp/divisor;
     temp%=divisor;
     rem=temp;
     temp=(temp*10) + num[i];

     flag=1;
  }
  else
  {
     rem=temp;
     temp=(temp*10) + num[i];

     if(flag==1)
     ans[k++]=0;
  }
 }

 for(i=0; i<k; i++)
 ans[i]+=48;
 ans[i]=0;

 printf("Quotient is: %s", ans);
 printf("\nRemainder is: %llu",rem);
}
```

a> Addition
b> Subtraction
c> Multiplication
d> Division
Enter your choice
a

Addition
Enter 1st Number: 45
Enter 2nd Number: 89
Answer is: 134

- **Subtraction :**

```
#include<stdio.h>
int main(){
  char num1[51], num2[51];
  int intNum1[50]={0}, intNum2[50]={0};
  int ans[51];

  printf("Enter large Number: ");
  scanf("%s", num1);
  printf("Enter small Number: ");
  scanf("%s", num2);

  int i=0, j=0, len1, len2, length;
  while(num1[i]) i++; i--;  // length of the 1st string
  while(num2[j]) j++; j--;  // length of the 2nd string
  len1=i;
  len2=j;
  if(len1>len2) length=len1;
  else length=len2;

  for(i=0, j=len1; i<=len1; i++, j--)  // reverse 1st
number
  intNum1[j]=num1[i]-48;

 for(i=0, j=len2; i<=len2; i++, j--)  // reverse 2nd
number
  intNum2[j]=num2[i]-48;

 int borrow=0;
 for(i=0; i<=length; i++){
   if(intNum1[i]>=intNum2[i]){
   ans[i]=intNum1[i] - (intNum2[i]+borrow);
   borrow=0;
   }

   else{
      intNum1[i]+=10;
      ans[i]=intNum1[i] - (intNum2[i]+borrow);
      borrow=1;
   }
 }
 for(i=length; i>=0; i--)  //eliminate the leading
zeros
 if(ans[i] !=0)
 break;

 for(; i>=0; i--)
 printf("%d",ans[i]);
}
```

Input :	**Output :**
Enter large Number: 456254 Enter small Number: 12422	443832

- **Division :**

```
#include<stdio.h>
int main(){
  char num[51];
  char ans[51];
  long long divisor;

  printf("Enter Divident Number: ");
  scanf("%s", num);
  printf("Enter Divisor Number: ");
  scanf("%llu", &divisor);

  int i=0, len;
  while(num[i]) i++; i--;  // length of the string
  len=i;

  for(i=0; i<=len; i++)  // string to integer conversion
  num[i]-=48;

  long long rem, temp;
  int k=0, flag=0;

  temp=num[0];
  rem=temp;
  for(i=1; i<=(len+1); i++){
    if(temp>=divisor){
       ans[k++]=temp/divisor;
       temp%=divisor;
       rem=temp;
       temp=(temp*10) + num[i];

       flag=1;
    }
    else {
       rem=temp;
       temp=(temp*10) + num[i];

       if(flag==1)
       ans[k++]=0;
    }
  }

  for(i=0; i<k; i++)
  ans[i]+=48;
  ans[i]=0;

  printf("Quotient is: %s", ans);
  printf("\nRemainder is: %llu",rem);
}
```

Input :	**Output :**
Enter Divident Number: 458 Enter Divisor Number: 4	Quotient is: 114 Remainder is: 2

56. Create a link list for the following structure and get user inputs of id, Maths mark, Science Mark. Once the user inputs id as -1 exit the input mode and display all the entries.

```
#include <stdio.h>
#include <stdlib.h>

struct student{
  int id;
  int maths;
  int science;
  struct student *next;
};

int main(){
 struct student *head = (struct student
*)malloc(sizeof(struct student));

 printf("Enter the id: ");
 scanf("%d",&head->id);
 printf("Enter math marks: ");
 scanf("%d",&head->maths);
 printf("Enter science marks: ");
 scanf("%d",&head->science);

 head->next=NULL;

 if(head->id==1){
  printf("\n\nid: %d",head->id);
  printf("\nmaths: %d", head->maths);
  printf("\nscience: %d", head->science);
 }
 return 0;
 }
```

Input :
Enter the id: 1
Enter math marks: 56
Enter science marks: 63

Output :
id: 1
math: 56
science: 63

Input :
Enter the id: 5
Enter math marks: 56
Enter science marks: 63

Output :

No Results

- **Basic Linked List :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

void linkListTraversal(struct node *ptr)
{
  while(ptr != NULL){
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 }
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = NULL;
    printf("Linked list ");
    linkListTraversal(head);
   return 0;
}
```

Output :

Linked list
data: 1
data: 2
data: 3

- **Circular Linked List :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

void linkListTraversal(struct node *head){
  struct node *p = (struct student *)malloc(sizeof(struct node));
  p=head;

  do{
    printf("\ndata: %d", p->data);
    p = p->next;
  }
  while(p != head);
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;
    struct node *fourth;

    head = (struct student *)malloc(sizeof(struct node));
    second = (struct student *)malloc(sizeof(struct node));
    third = (struct student *)malloc(sizeof(struct node));
    fourth = (struct student *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = fourth;
    fourth->data = 4;
    fourth->next = head;

    printf("Linked list ");
    linkListTraversal(head);
   return 0;
}
```

Output : Linked list

```
data: 1
data: 2
data: 3
data: 4
```

- **Link List Deletion 1 :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};
struct node *deleteFirst(struct node *head){
  struct node *p=(struct node *)malloc(sizeof(struct node));
  p=head;
  head=head->next;
  free(p);
 return head;
}
void linkListTraversal(struct node *ptr){
  while(ptr != NULL){
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 } }
int main() {
    struct node *head;
    struct node *second;
    struct node *third;
    struct node *fourth;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));
    fourth = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = fourth;

    fourth->data = 4;
    fourth->next = NULL;

    printf("Linked list before deletion ");
    linkListTraversal(head);

    head=deleteFirst(head);
    printf("\n\nLinked list after deletion ");
    linkListTraversal(head);

   return 0;
}
```

- **Link List Deletion 2 :**

```
#include <stdio.h>                  //  (page – 1)
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *deleteIndex(struct node *head, int
index){
  struct node *p=(struct node *)malloc(sizeof(struct
node));
  struct node *q=(struct node *)malloc(sizeof(struct
node));
  p=head;
  int i=0;
  while(i != index-1){
    p=p->next;
    i++;
  }
  q=p->next;
  p->next=q->next;
  free(q);

 return head;
}

void linkListTraversal(struct node *ptr)
{
  while(ptr != NULL)
{
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 }
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;
    struct node *fourth;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));
    fourth = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = fourth;              //  (page – 2)

    fourth->data = 4;
    fourth->next = NULL;

    printf("Linked list before deletion ");
    linkListTraversal(head);

    head=deleteIndex(head, 2);
    printf("\n\nLinked list after deletion ");
    linkListTraversal(head);

   return 0;
}
```

- **Link List Deletion 3 :**

```
#include <stdio.h>               //  (page – 1)
#include <stdlib.h>

struct node{
 int data;
 struct node *next;
};

struct node *deleteLast(struct node *head){
 struct node *p=(struct node *)malloc(sizeof(struct node));
 struct node *q=(struct node *)malloc(sizeof(struct node));
 q=head;
 p=q->next;
 while(p->next != NULL)
 {
  p=p->next;
  q=q->next;
 }
 q->next=NULL;
 free(p);

 return head;
}

void linkListTraversal(struct node *ptr)
{
 while(ptr != NULL)
{
 printf("\ndata: %d", ptr->data);
 ptr = ptr->next;
 }
}

int main() {
  struct node *head;
  struct node *second;
  struct node *third;
  struct node *fourth;

  head = (struct node *)malloc(sizeof(struct node));
  second = (struct node *)malloc(sizeof(struct node));
  third = (struct node *)malloc(sizeof(struct node));
  fourth = (struct node *)malloc(sizeof(struct node));

  head->data = 1;
  head->next = second;

  second->data = 2;
  second->next = third;

  third->data = 3;
  third->next = fourth;

  fourth->data = 4;
  fourth->next = NULL;
```

```
  printf("Linked list before deletion ");
  linkListTraversal(head);              //  (page – 2)

  head=deleteLast(head);
  printf("\n\nLinked list after deletion ");
  linkListTraversal(head);

  return 0;
}
```

- **Link List Deletion 4 :**

```
#include <stdio.h>              //   (page – 1)
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *deleteNode(struct node *head, struct
node *second){
  struct node *p=(struct node *)malloc(sizeof(struct
node));
  struct node *q=(struct node *)malloc(sizeof(struct
node));
 q=head;
 p=q->next;
  while(p != second)
  {
    p=p->next;
    q=q->next;
  }
  q->next=p->next;
  free(p);

 return head;
}

void linkListTraversal(struct node *ptr)
{
  while(ptr != NULL)
{
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 }
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;
    struct node *fourth;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));
    fourth = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;              //   (page – 2)

    third->data = 3;
    third->next = fourth;

    fourth->data = 4;
    fourth->next = NULL;

    printf("Linked list before deletion ");
    linkListTraversal(head);

    head=deleteNode(head, second);
    printf("\n\nLinked list after deletion ");
    linkListTraversal(head);

   return 0;
}
```

- **Link List Insertion 1 :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *insertAtFirst(struct node *head, int data){
  struct node *ptr=(struct node *)malloc(sizeof(struct
node));
  ptr->next=head;
  ptr->data=data;
  return ptr;
}

void linkListTraversal(struct node *ptr)
{
  while(ptr != NULL)
{
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 }
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = NULL;
    printf("Linked list before intertion ");
    linkListTraversal(head);

    head=insertAtFirst(head, 56);
    printf("\nLinked list after intertion ");
    linkListTraversal(head);

   return 0;
}
```

- **Link List Insertion 2 :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *insertAtIndex(struct node *head, int data,
int index){
  struct node *ptr=(struct node *)malloc(sizeof(struct
node));
  struct node *p=head;
  int i=0;

  while(i != (index-1)){
  p=p->next;
  i++;
  }
  ptr->data=data;
  ptr->next=p->next;
  p->next=ptr;
  return head;
}
void linkListTraversal(struct node *ptr){
  while(ptr != NULL){
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 } }
int main() {
    struct node *head;
    struct node *second;
    struct node *third;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;
    second->data = 2;
    second->next = third;
    third->data = 3;
    third->next = NULL;
    printf("Linked list before intertion ");
    linkListTraversal(head);

    head=insertAtIndex(head, 56, 2);
    printf("\nLinked list after intertion ");
    linkListTraversal(head);
   return 0;
}
```

- **Link List Insertion 3 :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *insertAtLast(struct node *head, int data){
  struct node *ptr=(struct node *)malloc(sizeof(struct
node));
  struct node *p=head;

  while(p->next != NULL)
  p=p->next;

  ptr->data=data;
  ptr->next=NULL;
  p->next=ptr;
  return head;
}
void linkListTraversal(struct node *ptr){
  while(ptr != NULL){
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 } }
int main() {
    struct node *head;
    struct node *second;
    struct node *third;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = NULL;
    printf("Linked list before intertion ");
    linkListTraversal(head);

    head=insertAtLast(head, 56);
    printf("\nLinked list after intertion ");
    linkListTraversal(head);

   return 0;
}
```

- **Link List Insertion 4 :**

```
#include <stdio.h>
#include <stdlib.h>

struct node{
  int data;
  struct node *next;
};

struct node *insertAfterNode(struct node *node, int
data){
  struct node *ptr=(struct node *)malloc(sizeof(struct
node));

  ptr->data=data;
  ptr->next=node->next;
  node->next=ptr;

 // return head;
}

void linkListTraversal(struct node *ptr){
  while(ptr != NULL){
  printf("\ndata: %d", ptr->data);
  ptr = ptr->next;
 }
}

int main() {
    struct node *head;
    struct node *second;
    struct node *third;

    head = (struct node *)malloc(sizeof(struct node));
    second = (struct node *)malloc(sizeof(struct node));
    third = (struct node *)malloc(sizeof(struct node));

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = NULL;
    printf("Linked list before intertion ");
    linkListTraversal(head);

    insertAfterNode(second, 56);
    printf("\nLinked list after intertion ");
    linkListTraversal(head);
   return 0;
}
```

57. Create a sample link list with about 5 entries using the following structure. Insert a new entry before or after a given id.

Menu Items
1. Insert Entry
2. Display List
3. Exit

```
#include <stdio.h>                    //  (page – 1)
#include <stdlib.h>

struct student{
  int id;
  int maths;
  int science;
  struct student *next;
};

void insert(struct student *head, int idn)
{
  struct student *new = (struct student
*)malloc(sizeof(struct student));
  struct student *p;
  p=head;

  while(p->id != idn)
   p=p->next;

  new->next=p->next;
  p->next=new;

 printf("\nEnter the new id: ");
 scanf("%d",&new->id);
 printf("Enter math marks: ");
 scanf("%d",&new->maths);
 printf("Enter science marks: ");
 scanf("%d",&new->science);
}

void scan(struct student *head)
{
 struct student *ptr = head;
while(ptr != NULL){
 printf("\nEnter the id: ");
 scanf("%d",&ptr->id);
 printf("Enter math marks: ");
 scanf("%d",&ptr->maths);
 printf("Enter science marks: ");
 scanf("%d",&ptr->science);
 ptr=ptr->next;
 }
}
```

```
void print(struct student *head){
  struct student *ptr = head;              //  (page – 2)
  while(ptr != NULL){
  printf("\n\nid: %d",ptr->id);
  printf("\nmaths: %d", ptr->maths);
  printf("\nscience: %d", ptr->science);
  ptr=ptr->next;
  }
}
int main()
{
 struct student *head = (struct student
*)malloc(sizeof(struct student));
 struct student *second = (struct student
*)malloc(sizeof(struct student));
 struct student *third = (struct student
*)malloc(sizeof(struct student));
 struct student *fourth = (struct student
*)malloc(sizeof(struct student));
 struct student *fifth = (struct student
*)malloc(sizeof(struct student));

 head->next=second;
 second->next=third;
 third->next=fourth;
 fourth->next=fifth;
 fifth->next=NULL;

 scan(head);
 printf("\nEntries before Insertion");
 print(head);

 int ins_id;
 printf("\n\nInsertion of new after the id no: ");
 scanf("%d",&ins_id);
 insert(head,ins_id);
 printf("\n\nEntries after Insertion");
 print(head);
}
```

58. Create a sample linked list with about 5 entries using the following structure. Insert a new entry before or after a given id. Delete an entry of a given id.

Menu Items
1. Insert Entry
2. Delete Entry
3. Display List
4. Exit

```
#include <stdio.h>
#include <stdlib.h>

struct student{
  int id;
  int maths;
  int science;
  struct student *next;
};

void insert(struct student *head, int idn){
  struct student *new = (struct student
*)malloc(sizeof(struct student));
  struct student *p;
  p=head;

  while(p->id != idn)
   p=p->next;

  new->next=p->next;
  p->next=new;

 printf("\nEnter the new id: ");
 scanf("%d",&new->id);
 printf("Enter math marks: ");
 scanf("%d",&new->maths);
 printf("Enter science marks: ");
 scanf("%d",&new->science);
}

struct student *delete(struct student *head, int id){
 struct student *p, *q;
 p=head;

if(id==1){
    head=head->next;
    free(p);
 }

else{
 while(p->id != (id-1))
  p=p->next;

 q=p->next;
 p->next = q->next;
 free(q);
 }
return head;
}

void scan(struct student *head){
 struct student *ptr = head;
while(ptr != NULL){
 printf("\nEnter the id: ");
 scanf("%d",&ptr->id);
 printf("Enter math marks: ");
 scanf("%d",&ptr->maths);
 printf("Enter science marks: ");
 scanf("%d",&ptr->science);
 ptr=ptr->next;
 } }

void print(struct student *head){
  struct student *ptr = head;
  while(ptr != NULL){
  printf("\n\nid: %d",ptr->id);
  printf("\nmaths: %d", ptr->maths);
  printf("\nscience: %d", ptr->science);
  ptr=ptr->next;
  } }

int main(){
 struct student *head = (struct student
*)malloc(sizeof(struct student));
 struct student *second = (struct student
*)malloc(sizeof(struct student));
 struct student *third = (struct student
*)malloc(sizeof(struct student));
 struct student *fourth = (struct student
*)malloc(sizeof(struct student));
 struct student *fifth = (struct student *)malloc(sizeof(struct
student));

 head->next=second;
 second->next=third;
 third->next=fourth;
 fourth->next=fifth;
 fifth->next=NULL;

 scan(head);
 printf("\nEntries before Insertion");
 print(head);

 int ins_id;
 printf("\n\nInsertion of new after the id no: ");
 scanf("%d",&ins_id);
 insert(head,ins_id);
 printf("\n\nEntries after Insertion");
 print(head);

 int del_id;
 printf("\n\nEnter id no. of deletion: ");
 scanf("%d",&del_id);
 head = delete(head,del_id);
 printf("\n\nEntries after Deletion");
 print(head);
}
```

59. Create a doubly linked list with about 5 entries using the following structure. Insert a new entry before or after a given id. Delete an entry of a given id.

Menu Items
1. Insert Entry
2. Delete Entry
3. Display List
4. Exit

```
#include <stdio.h>                // (page – 1)
#include <stdlib.h>

struct student{
  struct student *prev;
  int id;
  int maths;
  int science;
  struct student *next;
};

void insert(struct student *head, int idn){
  struct student *new = (struct student
*)malloc(sizeof(struct student));
  struct student *p, *q;
  p=head;
  q=p->next;

  while(p->id != idn){
   p=p->next;
   q=q->next;
  }

  new->next=q;
  p->next=new;

  new->prev=p;
  q->prev=new;

 printf("\nEnter the new id: ");
 scanf("%d",&new->id);
 printf("Enter math marks: ");
 scanf("%d",&new->maths);
 printf("Enter science marks: ");
 scanf("%d",&new->science);
}

struct student *delete(struct student *head, int id){
 struct student *p, *q, *r;
 p=head;

if(id==1){
   head=head->next;
   head->prev=NULL;
   free(p);
 }
else{
 while(p->id != (id-1))           // (page – 2)
  p=p->next;

 q=p->next;
 r=q->next;

 p->next = r;
 r->prev=p;

 free(q);
 }
 return head;
}

void scan(struct student *head){
 struct student *ptr = head;
while(ptr != NULL){
 printf("\nEnter the id: ");
 scanf("%d",&ptr->id);
 printf("Enter math marks: ");
 scanf("%d",&ptr->maths);
 printf("Enter science marks: ");
 scanf("%d",&ptr->science);
 ptr=ptr->next;
 }
}
void print(struct student *head){
  struct student *ptr = head;
  while(ptr != NULL){
  printf("\n\nid: %d",ptr->id);
  printf("\nmaths: %d", ptr->maths);
  printf("\nscience: %d", ptr->science);
  ptr=ptr->next;
  }
}
int main(){
 struct student *head = (struct student
*)malloc(sizeof(struct student));
 struct student *second = (struct student
*)malloc(sizeof(struct student));
 struct student *third = (struct student
*)malloc(sizeof(struct student));
 struct student *fourth = (struct student
*)malloc(sizeof(struct student));
 struct student *fifth = (struct student *)malloc(sizeof(struct
student));

 head->next=second;
 second->next=third;
 third->next=fourth;
 fourth->next=fifth;
 fifth->next=NULL;

head->prev=NULL;
second->prev=head;
third->prev=second;
fourth->prev=third;
fifth->prev=fourth;
```

```
scan(head);                    //  (page – 3)
printf("\nEntries before Insertion");
print(head);

int ins_id;
printf("\n\nInsertion of new after the id no: ");
scanf("%d",&ins_id);
insert(head,ins_id);
printf("\n\nEntries after Insertion");
print(head);

int del_id;
printf("\n\nEnter id no. of deletion: ");
scanf("%d",&del_id);
head = delete(head,del_id);
printf("\n\nEntries after Deletion");
print(head);
}
```

60. Write a program create a stack using linked list. Use push & Pop. Push will insert the entry in top of the list. Pop will get top of the list and display. Display will show from top to bottom.

Menu Items
1. Push
2. Pop
3. Display Stack
4. Exit

```
#include <stdio.h>             //  (page – 1)
#include <stdlib.h>

struct student{
  int id;
  int maths;
  int science;
  struct student *next;
};

int isEmpty(struct student *top)
{
 if(top==NULL)
    return 1;

    else return 0;
}

int isFull()
{
 struct student *p = (struct student *)malloc(sizeof(struct
student));
 if(p==NULL)
    return 1;

  else return 0;
}

void scan(struct student *ptr)
{
 printf("\nEnter the id: ");
 scanf("%d",&ptr->id);
 printf("Enter math marks: ");
 scanf("%d",&ptr->maths);
 printf("Enter science marks: ");
 scanf("%d",&ptr->science);
}

void print(struct student *top)
{
  if(isEmpty(top))
  printf("\nEmpty");
```

```
 struct student *ptr = top;              //   (page – 2)
 while(ptr != NULL){
 printf("\n\nid: %d",ptr->id);
 printf("\nmaths: %d", ptr->maths);
 printf("\nscience: %d", ptr->science);
 ptr=ptr->next;
 }
}

struct student *push(struct student *top)
{
 if(isFull())
 printf("Memory Full");

 else{
 struct student *new = (struct student *)malloc(sizeof(struct
student));
 new->next=top;
 top=new;
 scan(top);
 }
 return top;
}

struct student *pop(struct student *top)
{
  if(isEmpty(top))
  printf("\nEmpty");

  else{
    struct student *p;
    p=top;
    top=top->next;
    free(p);
  }
  return top;
}

int main()
{
  int x;
  struct student *top = NULL;

loop:
 printf("\n\n\n1> Push");
 printf("\n2> Pop");
 printf("\n3> Display");
 printf("\n4> Exit");
 printf("\nEnter your choice: ");
 scanf("%d", &x);
```

```
switch(x)              //   (page – 3)
 {
  case 1: top = push(top);
          break;
  case 2: top = pop(top);
          break;
  case 3: print(top);
          break;
  case 4: return 0;
          break;
  default: printf("\nInvalid Input");
           break;
 }

 goto loop;
}
```

61. Write a program to create a queue using linked list. Use add and remove. Add will insert the entry in top of the list. Remove will get bottom of the list and display. Display will show from top to bottom.

Menu Items
1. Add
2. Remove
3. Display Stack
4. Exit

```
#include <stdio.h>            //  (page – 1)
#include <stdlib.h>

struct student{
  int id;
  int maths;
  int science;
  struct student *next;
};

int isEmpty(struct student *top){
 if(top==NULL)
    return 1;

    else return 0;
}

int isFull(){
 struct student *p = (struct student *)malloc(sizeof(struct student));
 if(p==NULL)
    return 1;

  else return 0;
}

void scan(struct student *ptr){
 printf("\nEnter the id: ");
 scanf("%d",&ptr->id);
 printf("Enter math marks: ");
 scanf("%d",&ptr->maths);
 printf("Enter science marks: ");
 scanf("%d",&ptr->science);
}

void print(struct student *top){
  if(isEmpty(top))
  printf("\nEmpty");

  struct student *ptr = top;
  while(ptr != NULL){
  printf("\n\nid: %d",ptr->id);
  printf("\nmaths: %d", ptr->maths);
  printf("\nscience: %d", ptr->science);
  ptr=ptr->next;
  }
}

struct student *add(struct student *top){
 if(isFull())
 printf("Memory Full");            //  (page – 2)

 else{
 struct student *new = (struct student *)malloc(sizeof(struct student));
 new->next=top;
 top=new;
 scan(top);
 }
 return top;
}

struct student *remov(struct student *top){
  if(isEmpty(top))
  printf("\nEmpty");

  else{
    struct student *p, *q;
    p=top;
    q=p->next;

    if(p->next==NULL){
       top=NULL;
       free(p);
    }

   else{
    while(q->next != NULL)
    {
     q=q->next;
     p=p->next;
    }

   p->next=NULL;
   free(q);
   }
  }
  return top;
}

int main(){
  int x;
  struct student *top = NULL;

loop:
 printf("\n\n\n1> Add");
 printf("\n2> Remove");
 printf("\n3> Display");
 printf("\n4> Exit");
 printf("\nEnter your choice: ");
 scanf("%d", &x);
```

```
switch(x)
{
 case 1: top = add(top);           //   (page – 3)
        break;
 case 2: top = remov(top);
        break;
 case 3: print(top);
        break;
 case 4: return 0;
        break;
 default: printf("\nInvalid Input");
         break;
}

goto loop;
}
```